

PE-means: Improved Differentially Private k -means Clustering through Private Evolution

Thomas Humphries
University of Waterloo
thomas.humphries@uwaterloo.ca

Zinan Lin
Microsoft Research
zinanlin@microsoft.com

Sergey Yekhanin
Microsoft Research
yekhanin@microsoft.com

ABSTRACT

We study the problem of differentially private (DP) k -means clustering in Euclidean space. Previous solutions rely on summing the private data directly, which induces a sensitivity proportional to the domain. We introduce PE-means, an extension of the private evolution (PE) algorithm (an increasingly popular method for synthetic data generation), to the problem of k -means clustering. The key advantage of PE is that it only computes a private histogram with constant sensitivity to guide the evolution. Our adaptation of PE includes new evolutionary operators for clustering, as well as other algorithmic improvements of independent interest. Overall, PE-means achieves an average improvement of 26% in clustering loss over state-of-the-art baselines such as Google’s LSH-based algorithm and DP-Lloyd variants.

1 INTRODUCTION

The k -means clustering algorithm is a foundational tool in data science with many influential applications such as recommendation systems and healthcare data analysis [16, 25]. The objective is to group the data into k groups with similar features, allowing practitioners to interpret complex datasets. The challenge is that applying standard k -means algorithms to sensitive data risks the privacy of participants in the dataset. In the worst case, the k -means algorithm can publish the exact data record of an individual who is an outlier in the set. To address this, there has been a large body of work that considers the problem of differentially private k -means [1, 3–5, 12, 15, 25, 30, 31]. Differential privacy (DP) is a popular privacy definition that ensures a change in a single user’s input does not have a significant effect on the output of the mechanism [10]. By adding calibrated randomization to the clustering process, existing approaches retain the benefits of k -means clustering while providing a formal privacy guarantee.

The challenge with DP is balancing the inherent trade-off between privacy and utility. Existing state-of-the-art approaches for DP k -means rely on summing the private data directly at some point in the process [1, 4, 31]. The sensitivity of (or largest impact any one user can have on) a sum query is determined by the addition (or removal) of a point on the boundary of the domain. This means that to satisfy DP, noise must be added proportionally to the entire domain, significantly degrading the quality of the clustering. In recent work, an approach called FastLloyd managed to reduce this sensitivity from the whole domain to a smaller radius around each cluster through relative cluster updates [5]. However, the radius is still proportional to the data dimension, and FastLloyd introduces an additional error term due to the relative updates.

Private evolution (PE) is an increasingly popular technique for generating various modalities of synthetic data [21, 22, 32, 33]. PE evolves a population of synthetic data using only inference API

access to foundation models that generate and perturb the data in an evolutionary algorithm. A key component in PE’s success is that it only uses the private data in a nearest neighbour voting scheme to choose the best samples generated so far. Specifically, each private data point only contributes a single vote to the algorithm in each iteration. Thus, adding or removing any private data point affects the output by a constant sensitivity of 1, regardless of the data domain. This gives PE a significant advantage over gradient-based techniques, which must add noise proportionally to the gradient domain. What makes PE truly impressive is that, despite getting much less signal from the private data, it still effectively traverses large and complex data domains by leveraging the robust optimization characteristics of evolution.

In this work, we introduce PE-means, an extension of the PE framework to the problem of k -means clustering. In this application, the population of PE contains randomly generated centroids that are iteratively evolved towards the centroids of the private data. Our design replaces PE’s API calls to foundation models with a lightweight clustering initialization and a Lévy flight-based mutation operator. In addition to adapting PE to clustering, we also make several improvements to the algorithm that are of independent interest. Specifically, we address a critical issue with PE’s selection technique where votes can be split between the best candidates, causing no representative from certain areas to be selected. We also reduce the impact of DP noise through better post-processing of the vote histogram and a mechanism to adaptively adjust the signal-to-noise ratio at no cost to privacy. Our experimental evaluation shows PE-means outperforms state-of-the-art baselines over numerous real and synthetic datasets. Furthermore, we propose HDPE-means, a variant of PE-means that uses a similar dimensionality reduction to Balcan et al.’s [1], allowing PE-means to scale to higher dimensions. Overall, we observe up to 91% improvement in clustering loss over the state-of-the-art, with an average improvement of 26% across all the datasets evaluated.

The rest of the paper is organized as follows. We first give the necessary background in Section 2. Section 3 details our design through a step-by-step treatment of the PE algorithm. In Section 4, we provide all experimental evidence including a case study on vote-splitting, the tuning of hyperparameters, and a utility evaluation of PE-means and HDPE-means. Finally, Section 5 surveys the relevant literature before concluding in Section 6.

2 BACKGROUND

2.1 k -Means Clustering

Given a dataset D of size N and dimension d , the k -means problem aims to partition the dataset into k groups (clusters) $C = \{C_1, \dots, C_k\}$, with each $C_i \subset D$, such that the following objective

is minimized,

$$\arg \min_C \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2 \quad (1)$$

where μ_i is the centroid of cluster i ($\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$). In this work, we assume the size and dimension of the dataset are public information, but we require any other information published from the dataset to satisfy differential privacy.

2.2 Differential Privacy

Differential privacy (DP) [10] adds randomness to the computation of aggregate statistics to protect the privacy of individuals. DP guarantees that an algorithm's output is approximately the same, regardless of the participation of any one user. More formally, differential privacy can be defined as follows.

DEFINITION 2.1 (DIFFERENTIAL PRIVACY). A randomized algorithm $M : \mathcal{D} \mapsto \mathbb{R}$ is (ϵ, δ) -DP, if for any pair of neighbouring datasets $D, D' \in \mathcal{D}$, and for any $S \subseteq \mathbb{R}$ we have

$$\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S] + \delta. \quad (2)$$

The privacy parameter ϵ defines how similar the output distributions must be, and δ allows a small chance of failure in the definition. We use the unbounded neighbouring definition, where datasets are neighbours if $|D \setminus D' \cup D' \setminus D| = 1$ (we allow for the addition or removal of a single data point). Arbitrary computations can be carried out on the output of a DP mechanism without affecting privacy due to the post-processing lemma [11]. Finally, DP is composed naturally with multiple runs of a mechanism. If we apply differentially private mechanisms sequentially, the privacy parameter composes through summation or more advanced methods [11].

DEFINITION 2.2 (SENSITIVITY). Let $f : \mathcal{D} \mapsto \mathbb{R}^k$. If $\mathbb{D}$ is a distance metric between elements of $\mathbb{R}^k$, then the $\mathbb{D}$ -sensitivity of f is

$$\Delta^{(f)} = \max_{(D, D')} \mathbb{D}(f(D), f(D')), \quad (3)$$

where (D, D') are pairs of neighbouring datasets.

We primarily use the ℓ_2 norm as the distance metric $\mathbb{D}$. To satisfy DP, we use Gaussian noise in this work and analyze it with Gaussian Differential Privacy (GDP) [6].

DEFINITION 2.3 (GDP [6]). A mechanism M is said to satisfy θ -Gaussian Differential Privacy (θ -GDP) if

$$\mathcal{T}(M(D), M(D')) \geq G_\theta$$

for all neighbouring datasets D and D' , where $\mathcal{T}$ is a trade-off function measuring the difficulty for attackers in identifying the presence of an individual data point and $G_\theta = \mathcal{T}(\mathcal{N}(0, 1), \mathcal{N}(\theta, 1))$ (see Dong et al. [6] for specifics of the definition).

Naturally, the Gaussian Mechanism satisfies GDP.

THEOREM 2.1. (Gaussian Mechanism for GDP [6]) Define the Gaussian mechanism that operates on a statistic f as $M(D) = f(D) + \eta$, where $\eta \sim \mathcal{N}(0, (\Delta^{(f)})/\theta)$. Then, M is θ -GDP.

We consider the composition of GDP over multiple runs of a mechanism.

Algorithm 1 PE-means based on Private Evolution (PE) [22]

Input: Private samples: $D = \{x_i\}_{i=1}^N \subset \mathbb{R}^d$
 Number of iterations: T
 Number of generated samples: $N_{\text{syn}} = k$
 Number of initial variations: L
 Noise multiplier for the DP Nearest Neighbors Histogram: σ
Output: DP cluster centres $S'_T = \{\mu_1, \dots, \mu_k\} \subset \mathbb{R}^d$

```

1:  $S_0 \leftarrow \text{RANDOM\_API}(N_{\text{syn}} * (L + 1))$ 
2: for  $t \leftarrow 1, \dots, T$  do
3:    $\text{hist}_t \leftarrow \text{DP\_NN\_HISTOGRAM}(D, S_{t-1}, \sigma) \triangleright \text{Algorithm 2}$ 
4:    $S'_t \leftarrow \text{select top-}N_{\text{syn}} \text{ samples from } S_{t-1} \text{ ranked by } \text{hist}_t$ 
5:    $S'_t \leftarrow \text{WEIGHTED\_K\_MEANS}(S_{t-1}, k=N_{\text{syn}}, \text{weight}=\text{hist}_t)$ 
6:    $S_t \leftarrow S'_t \triangleright \text{Save best samples}$ 
7:   if  $\text{sum}(\text{hist}_t^2)/k(L+1)\sigma^2 < 1.0$  then
8:      $L \leftarrow L/2$ 
9:   end if
10:  for  $i \leftarrow 1, \dots, L$  do
11:     $S_t \leftarrow S_t \cup \text{VARIATION\_API}(S'_i)$ 
12:  end for
13: end for
14: return  $S'_T$ 
```

THEOREM 2.2 (GDP COMPOSITION [6]). The n -fold composition of θ_i -GDP mechanisms is $\sqrt{\theta_1^2 + \dots + \theta_n^2}$ -GDP.

We also have a way to convert between GDP and DP.

THEOREM 2.3 (GDP TO DP [2, 6]). A mechanism is θ -GDP if and only if it is $(\epsilon, \delta(\epsilon))$ -DP for all $\epsilon \geq 0$, where

$$\delta(\epsilon) = \Phi\left(-\frac{\epsilon}{\theta} + \frac{\theta}{2}\right) - e^\epsilon \Phi\left(-\frac{\epsilon}{\theta} - \frac{\theta}{2}\right).$$

In practice, we follow the implementation of PE¹ to solve this function for θ , given a desired ϵ and δ .

3 METHODOLOGY

Private Evolution (PE) is an emerging private synthetic data generation technique that evolves a set of synthetic samples using only inference API access to large machine learning models [21, 22, 32, 33]. At a high level, PE starts from randomly generated synthetic data (using the RANDOM_API) and iteratively evolves toward high-quality synthetic data by creating new variations (using the VARIATION_API) and ranking the synthetic data that is closest to the private data. We build upon the text variant of PE called AugPE [33]. In Algorithm 1, we outline the baseline version of PE. Algorithm 1 also illustrates our approach, PE-means, highlighting our modifications to both improve PE and adapt it to solve k -means clustering. In this section, we detail the design of PE-means with respect to each part of the algorithm. Following this, we present our high-dimensional variant of PE-means in Section 3.5 and prove the privacy of both algorithms in Section 3.6.

¹<https://github.com/microsoft/DPSDA>

Algorithm 2 Modified DP_NN_HISTOGRAM

Input: Private samples: D
 Generated samples: $S = \{z_i\}_{i=1}^n$
 Noise multiplier: σ
 Distance function: $d(\cdot, \cdot)$
Output: DP nearest neighbours histogram on S

```

1:  $histogram \leftarrow [0, \dots, 0]$ 
2: for  $x_{priv} \in D$  do
3:    $i = \arg \min_{j \in [n]} d(x_{priv}, z_j)$ 
4:    $histogram[i] \leftarrow histogram[i] + 1$ 
5: end for
6:  $histogram \leftarrow histogram + N(0, \sigma I_n)$ 
7:  $histogram \leftarrow \text{sort\_descending}(histogram)$ 
8:  $k \leftarrow \min \{ \kappa \mid \sum_{j=1}^{\kappa} histogram[j] > N \}$ 
9:  $histogram[j] \leftarrow 0$  for all  $j > k$   $\triangleright$  Zero out noisy values
10: return  $histogram$ 

```

3.1 Random API

The first step in PE is to generate an initial population, independently of the private data, using the RANDOM_API (Line 1). In the earliest variants of PE [22, 33], this is done by querying a pre-trained foundation model (using information such as the label, which is assumed to be public). To solve clustering with PE, rather than creating a population of synthetic data samples, we create a population of centroids to evolve. We assume the only context available to generate the initial population is the shape of the data domain. For simplicity, we assume the domain is a hyper-sphere and use the same ℓ_2 bound R on its radius that other approaches need to bound sensitivity [1, 4, 5, 31]. However, in our case, the bound being too tight or loose can at most affect convergence speed and has no effect on the amount of noise added. A naive RANDOM_API for clustering would simply generate uniformly random data within the hyper-sphere. However, random points may group together, which reduces our coverage of the domain. Thus, we instead follow the work of Su et al. [31] and use the sphere packing method. Su et al.’s method starts with a given radius a (typically half the radius of the domain), and iteratively adds random samples that are at least a away from the boundary and $2a$ from the samples added so far. If, after a fixed number of retries, a random sample cannot be found to meet these conditions, a is halved and the algorithm continues until k samples are found. We adapt Su et al.’s algorithm to consider an ℓ_2 bounded hyper-sphere rather than an ℓ_∞ bounded hypercube. The rest of the algorithm remains the same.

3.2 Selection Step

The next step in PE is to select the best candidates from the population (either from the random initialization or previous iteration). First, the selection computes a private histogram of votes, then selects candidates based on these votes. We detail our modifications to both parts.

Histogram Computation. To utilize the private data in the selection, we compute the DP_NN_HISTOGRAM (Line 3) described in Algorithm 2. The idea is that all private samples vote for their

nearest neighbour in the population (Lines 3 and 4). This creates a histogram of votes that is easy to privatize using Gaussian noise in Line 6. However, depending on the size of the population and progression of the evolution, this histogram can be quite sparse. Some of the previous PE versions [21, 22, 32] post-processed the histogram by zeroing out any noisy votes below a certain threshold. The challenge is how to choose the optimal threshold without spending privacy budget. We propose a modification based on the constraint that, before adding noise, the histogram of votes should be non-negative and sum to the size of the dataset (which we assume to be public). The maximum likelihood estimation (MLE) of the true votes given these constraints is found by projecting the noisy votes onto the set $\{x \in \mathbb{R} \mid x_i \geq 0, \sum_i x_i = N\}$. Duchi et al. [8] give an efficient algorithm for this projection by computing a subset of bins with the largest noisy counts such that after scaling and thresholding they sum to exactly N . For clustering, we do not need the sum constraint to be exact; a more critical issue is points far from the private data getting assigned a few false votes due to noise and pulling the centroids in the wrong direction. Thus, we give a simple approximation of the MLE algorithm that computes the minimum number of highest-count buckets that exceed the sum constraint (Line 8) and zeros out the remaining buckets (Line 9), without any additional scaling. Intuitively, we assume histogram buckets with large counts are likely to have dominated the noise.

Selecting Candidates. After computing the private histogram, the next evolutionary task is to choose the best candidates from the population to survive to the next iteration based on this histogram. Aug-PE takes the approach of ranking candidates based on the number of votes they received in the histogram (Line 4). The drawback to this approach is that choosing the most popular points does not always yield optimal clustering coverage due to what we call vote splitting. Consider a scenario where PE generates a point that is the nearest neighbour of many private dataset points. In the next iteration, PE may generate numerous new samples close to this point, causing the votes to be split between these new samples. Then, since all new points have a low count, an entire area of feature space may not be selected. We illustrate an example of this for clustering in Section 4.2. We argue that instead of just choosing points with the highest votes, we also need to choose one or two solutions from areas with highly split votes. Our key observation is that standard k -means clustering algorithms have a similar objective. Thus, we use the population as a coreset weighted by the histogram and run standard k -means, using the resulting centroids as the selected set for the next round (Line 5).² We show that this technique is much more robust to vote splitting in Section 4.2.

3.3 Variation API

After selecting the best candidates, the next step is to continue exploring the space for better centroids by creating variations of the selected candidates. We note that we preserve the best candidates (referred to as elitism in evolutionary algorithms) in Line 6, so that if no variation improves, we maintain the current progress. In some of

²We could instead use a k -medians algorithm to select candidates strictly from the population satisfying a similar objective. However, we see no reason to enforce that constraint in this work, as the standard k -means algorithm can move us closer to the true centroids while making the selection.

the previous versions of PE, the VARIATION_API (Line 11) would be another API call to a foundation model. For example, Aug-PE [33] would randomly mask words and ask a large language model to fill them in with a high temperature. In the case of clustering, our VARIATION_API perturbs the real-valued candidate centroids directly. We find that mutation based on Lévy flights works well, as the distribution is heavier-tailed to encourage exploration. We follow Mantegna’s algorithm [24] to generate a Lévy stable distribution parameterized by the Lévy index β which we denote by $\text{Lévy}(\beta)$. To sample a value $\zeta \sim \text{Lévy}(\beta)$, we first compute

$$\sigma_u = \left(\frac{\Gamma(1+\beta) \sin\left(\frac{\pi\beta}{2}\right)}{\Gamma\left(\frac{1+\beta}{2}\right) \cdot \beta \cdot 2^{(\beta-1)/2}} \right)^{1/\beta}, \quad (4)$$

then draw independent samples $u \sim \mathcal{N}(0, \sigma_u I_d)$ and $v \sim \mathcal{N}(0, I_d)$ that we combine to get $\zeta = \frac{u}{|v|^{1/\beta}}$. Our variation API adds i.i.d. noise from this distribution to the existing centroids, scaled by a hyperparameter γ and the radius R , namely

$$\text{VARIATION_API}(\mu_i) = \mu_i + \gamma R \zeta \quad (5)$$

where $\zeta \sim \text{Lévy}(\beta)$. Finally, we clip any mutated sample with ℓ_2 norm greater than R to the boundary. We experimentally set the hyperparameters for our variation API and validate the use of Lévy over Gaussian noise in Section 4.3.2.

3.4 Adaptive Population Size

Following Aug-PE, we apply the variation API L times to each sample to increase exploration (Line 11). Tuning the L parameter is critical. If L is too small, the algorithm converges slowly, needing more iterations and privacy budget. Alternatively, if L is too large, the histogram becomes sparse, resulting in a poor signal-to-noise ratio and poor-quality selections. While we can tune L as a hyperparameter (we show a heuristic for setting it in Section 4.3), the best setting for L varies greatly depending on the distribution of the private data and the current distribution of the candidates. To help avoid the more damaging case of a large L preventing meaningful selection, we add adaptive tuning of L during evolution. In Line 7, we estimate the signal-to-noise ratio (without using any privacy budget) by computing the ℓ_2 norm of the noisy histogram vs. the expected ℓ_2 norm of the noise. We argue that if this value is less than 1, there is likely more noise than signal, and we reduce L by half (Line 8). We experimented with different thresholds and amounts of reduction but found these settings work well in practice.

3.5 High-Dimensional Case

In high-dimensional domains (especially when k is small), variations become less effective due to the curse of dimensionality. To combat this, in Algorithm 3, we present a modified version of PE-means that is optimized for higher dimensions. We follow the work of Balcan et al. [1] and first use a Johnson-Lindenstrauss transform to project the private data to a low-dimensional space (Line 2). We then run PE-means as normal in this low-dimensional space in Line 3. To project the resulting cluster centres back to the original domain, we use noisy averaging. This can be thought of as a single iteration of DP Lloyd’s algorithm [31] in the original domain, initialized with the low-dimensional cluster assignments.

Algorithm 3 High-Dimensional PE-means (HDPE-means)

Input: Private samples: $D = \{x_i\}_{i=1}^N \subset \mathbb{R}^d$

Reduced dimension: p

Number of clusters: k

Number of PE iterations: T

Number of PE variations: L

Noise multiplier: σ

Domain radius: R

Output: DP cluster centres $S = \{\mu_1, \dots, \mu_k\} \subset \mathbb{R}^d$

```

1: Sample random projection matrix  $G \sim \mathcal{N}(0, 1)^{p \times d}$ 
2:  $\tilde{D} \leftarrow \{\tilde{x}_i\}_{i=1}^N$  where  $\tilde{x}_i = \frac{1}{\sqrt{d}} G x_i$       ▶ Project data to  $\mathbb{R}^p$ 
3:  $\{\tilde{\mu}_1, \dots, \tilde{\mu}_k\} \leftarrow \text{PE-means}(\tilde{D}, T-2, k, L, \sigma)$  ▶ Get centroids in  $\mathbb{R}^p$  using Algorithm 1
4: for  $i \leftarrow 1, \dots, N$  do
5:    $a_i \leftarrow \arg \min_{j \in \{1, \dots, k\}} \|\tilde{x}_i - \tilde{\mu}_j\|_2$       ▶ Assign labels in  $\mathbb{R}^d$ 
6: end for
7: for  $j \leftarrow 1, \dots, k$  do
8:    $z_j \leftarrow \sum_{i: a_i=j} x_i + \mathcal{N}(0, R\sigma I_n)$       ▶ Noisy sum in  $\mathbb{R}^d$ 
9:    $n_j \leftarrow \sum_{i: a_i=j} 1 + \mathcal{N}(0, \sigma I_n)$       ▶ Noisy count
10:   $\mu_j \leftarrow \frac{z_j}{n_j}$ 
11: end for
12: return  $S = \{\mu_1, \dots, \mu_k\}$ 

```

Specifically, all private data points are assigned a label using the low-dimensional centroids (Lines 4 to 6). Then we compute the sum and count of all private points for a given label to obtain the centroids in high-dimensional space (Lines 7 to 11). To ensure privacy, we add Gaussian noise to the sum and count of points in each cluster. We reduce the number of iterations for PE by two in Line 3, so we can use the same noise multiplier to noise the sum and count. We show in Section 4 that HDPE-means outperforms PE-means in higher dimensions. The disadvantage of this approach is that, similar to related work [1, 4], we add noise proportional to R (the radius of the data domain) during the projection back to high dimensions. However, most other approaches apply strictly more noise, by applying radius-proportional noise in each iteration. Our approach takes advantage of the low sensitivity of PE-means to fit the centroids accurately in low-dimensional space before conducting a single iteration of radius-proportional noise in the final iteration.³ In future work, we will investigate alternate ways to scale the variation API while reducing this dependency.

3.6 Privacy Analysis

The privacy analysis of PE-means follows from the original PE paper [22] as our changes do not affect noise addition in Algorithm 2 or use private data in any way. We restate the analysis here for completeness.

THEOREM 3.1. *PE-means as defined in Algorithm 1 satisfies (ϵ, δ) -DP.*

³We note that Google-LSH also only uses a single iteration of radius-proportional noise. However, their approach assigns cluster membership based on a single LSH partition of the space, whereas we iteratively optimize cluster membership with PE, yielding a more accurate clustering.

PROOF. We break the proof into the same steps as the original paper [22]:

- **Step 1: Bounding the sensitivity (Definition 2.2) of the DP Nearest Neighbors Histogram in Algorithm 2.** Each private sample only contributes one vote. If we add or remove one sample, the resulting histogram will change by 1 in the ℓ_2 norm. Therefore, the sensitivity is 1.
- **Step 2: Viewing each PE iteration as a Gaussian mechanism.** In Line 6 of Algorithm 2, we add i.i.d. Gaussian noise with standard deviation σ to each bin. This is the only part of the algorithm that touches the private dataset.
- **Step 3: Viewing the entire PE algorithm as T adaptive compositions of Gaussian mechanisms.** This holds because PE simply applies Algorithm 2 T times sequentially.
- **Step 4: Viewing the entire PE algorithm as one Gaussian mechanism with noise multiplier $\sigma/\sqrt{T}$.** Using the composition theorem (Theorem 2.2) from Dong et al. [6, Corollary 2], we get that T applications of a $1/\sigma$ -GDP algorithm are $\sqrt{T}/\sigma$ -GDP.
- **Step 5: Computing DP parameters ϵ and δ .** The problem is simply computing a σ such that (ϵ, δ) -DP iff $\sqrt{T}/\sigma$ -GDP, for which we apply the formula of Balle and Wang [2].

□

Our high-dimensional extension of PE includes additional use of the private data in the projection, and thus we state and prove its privacy.

THEOREM 3.2. *HDPE-means as defined in Algorithm 3 satisfies (ϵ, δ) -DP.*

PROOF.

- **Step 1: Composition accounting.** We first assume that PE is called (in Line 3) with two fewer iterations than the number of iterations σ was computed for. Applying Theorem 3.1, we get that the run of PE is private with two extra applications of a Gaussian mechanism with standard deviation of σ remaining.
- **Step 2: Assignment as post-processing.** After calling PE, the assignment step (Lines 4 to 6) uses the published centroids from the previous iteration (post-processing) to divide the dataset into clusters. We can then apply parallel composition over each of the clusters. Thus, we can focus on the privacy cost of a single cluster for the remainder of the proof.
- **Step 3: Privacy of the Gaussian mechanisms.** For each cluster, we apply the Gaussian mechanism twice. The first is to compute a sum which has an ℓ_2 sensitivity of R since adding or removing a point can at most add the largest possible vector in the domain. The second is to compute the count, which has an ℓ_2 sensitivity of 1, as a data point can be counted at most once. Applying Theorem 2.1, the result follows.

□

4 EXPERIMENTS

In this section, we provide a complete experimental evaluation of PE-means and HDPE-means. We begin by detailing the experimental setup in Section 4.1, followed by a case study highlighting the effect of vote splitting in practice in Section 4.2. Section 4.3 chooses and validates effective default hyperparameters for our approach. Finally, Section 4.4 gives a complete utility benchmark compared to the baselines with additional discussion in Section 4.5.

4.1 Experimental Setup

Datasets. We use a combination of real and synthetic datasets from various sources. Most of the real datasets as well as the G2 (Gaussian-based synthetic) datasets come from the clustering datasets repository [14], with some additional real datasets from the UCI machine learning repository [7]. For Birch2 [35], we take 25,000 random samples from the dataset of 100,000. Gas [7], Letter [7], and MNIST [20] are included for comparability with Chang and Kamath [4]. For MNIST, we train a LeNet5 model and use neural representations following Chang and Kamath [4]. The *scale* synthetic datasets from Diaa et al. [5] were generated with the `clusterGeneration` R package [29] and include a random level of cluster overlap. Finally, we include synthetic datasets generated with the `make_blobs` function from Sklearn that generates isotropic Gaussian clusters. Table 1 includes a complete list of datasets and their sizes.

Baselines. Our baselines are selected from state-of-the-art DP k -means algorithms with publicly available implementations. The work of Chang and Kamath (which we denote by *Google-LSH*) is perhaps the most popular approach [4]. Chang and Kamath use a locality-sensitive hash tree to create a private coreset, upon which the non-private k -means algorithm is applied. We modify their code to fit our assumption that the size of the dataset is public (requiring less noise). Because the Google-LSH method is one of the most recent works, we also evaluate the approaches from its evaluation. The first is IBM’s differential privacy library [17] (which we denote by *DP-Lib*) that implements an improved version of Su et al.’s algorithm [31]. Su et al. apply noise to each step of Lloyd’s clustering algorithm [23] with various optimizations such as the sphere packing initialization. The second is the work of Balcan et al. (which we denote by *lcm17*) that first projects the data to a low-dimensional space before recursively dividing the space into cubes. They then apply a k -medians-style swapping algorithm to choose the best centres before projecting the result back into the high-dimensional space through noisy averaging. We discovered several inconsistencies in the privacy analysis of Balcan et al.’s implementation, which we detail in the documentation of our implementation. We note that both DP-Lib and lcm17 work in the pure differential privacy model and thus, following Google-LSH [4], we evaluate them with $\delta = 0$. The final related work is a recent improvement over Su et al.’s algorithm by Diaa et al. [5], which we denote by *FastLloyd*. FastLloyd modifies Su et al.’s work by using Gaussian noise and computing cluster updates relative to the previous iteration, thereby reducing the sensitivity.

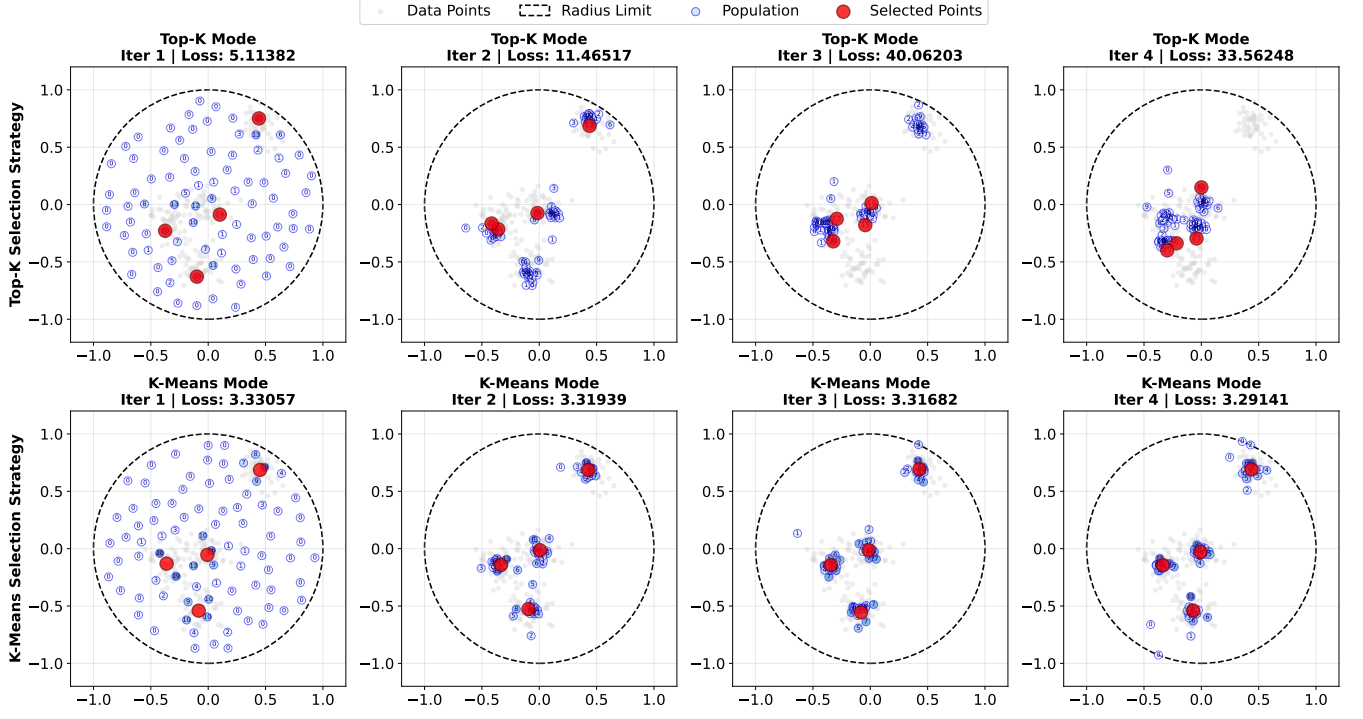


Figure 1: A visualization of the private data, samples generated, and samples selected by PE in each iteration, comparing the previous top- k selection approach to our proposed k -means approach with $\epsilon = \infty$. The k -means approach effectively mitigates the vote splitting problem and keeps selected points close to the true clusters.

Implementation Details. Our source code is available on GitHub.⁴ We normalize all datasets to have a max ℓ_2 norm of 1, for a clean presentation of metrics over different datasets. We follow Google-LSH and centre each dataset first by subtracting the mean [4]. Then, also following Google-LSH, we non-privately compute the ℓ_2 and ℓ_∞ sensitivity bounds on each dataset. In practice, this bound should be computed privately or derived from public information about the attributes. However, for the sake of comparison, we give all approaches an equal advantage by providing a tight bound on the domain. Each experiment is repeated 50 times over different random seeds, and we report the average. All shaded areas represent the 95% confidence interval of the mean of the results. We fix the privacy parameter $\delta = 1/N^{1.1}$, following the best practice that the failure probability should be less than $1/N$ [11]. The primary loss we consider is the normalized k -means loss.

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^N \min_{j=1}^k \|x_i - \mu_j\|^2 \quad (6)$$

We also compute each method’s performance over different privacy budgets using the area under the curve (AUC) of the loss values against $\epsilon \in \{0.25, 0.5, 1.0, 2.0, 4.0\}$, computed via the trapezoidal rule:

$$\text{AUC} = \sum_{i=1}^{n-1} \frac{\text{Loss}_i + \text{Loss}_{i+1}}{2} \cdot (\epsilon_{i+1} - \epsilon_i) \quad (7)$$

4.2 Vote Splitting Example

In Section 3.2, we described the issue of vote splitting in PE’s selection. In this section, we illustrate the phenomenon with a toy example. We generate a two-dimensional random dataset with $k = 4$ using the `make_blobs` function and set $\epsilon = \infty$. In Figure 1, we plot four iterations of PE (one per column) and show the difference between the previous top- k voting (top row of plots) and our weighted k -means selection (bottom row of plots). The plots include the private data in grey, the population of PE in blue, and the selected points for each method in red.

In the case of the top- k voting (first row), we observe that the initial iteration chooses points close to the four true clusters in the dataset. However, in the second iteration, there are many good candidates in the lower cluster, splitting the vote and causing all selected points to be in the other three clusters. Then, in the third iteration, the lower cluster dies out and the vote becomes split in the upper cluster. The clusters do not recover in the fourth iteration, as it will now take PE numerous iterations to move back towards those areas. In contrast, the bottom row of plots shows PE maintaining and refining four healthy clusters throughout the iterations. We note that in this toy example, PE has essentially converged in the first iteration, which further highlights the challenge with top- k voting.

⁴<https://github.com/t3humphries/PE-means>

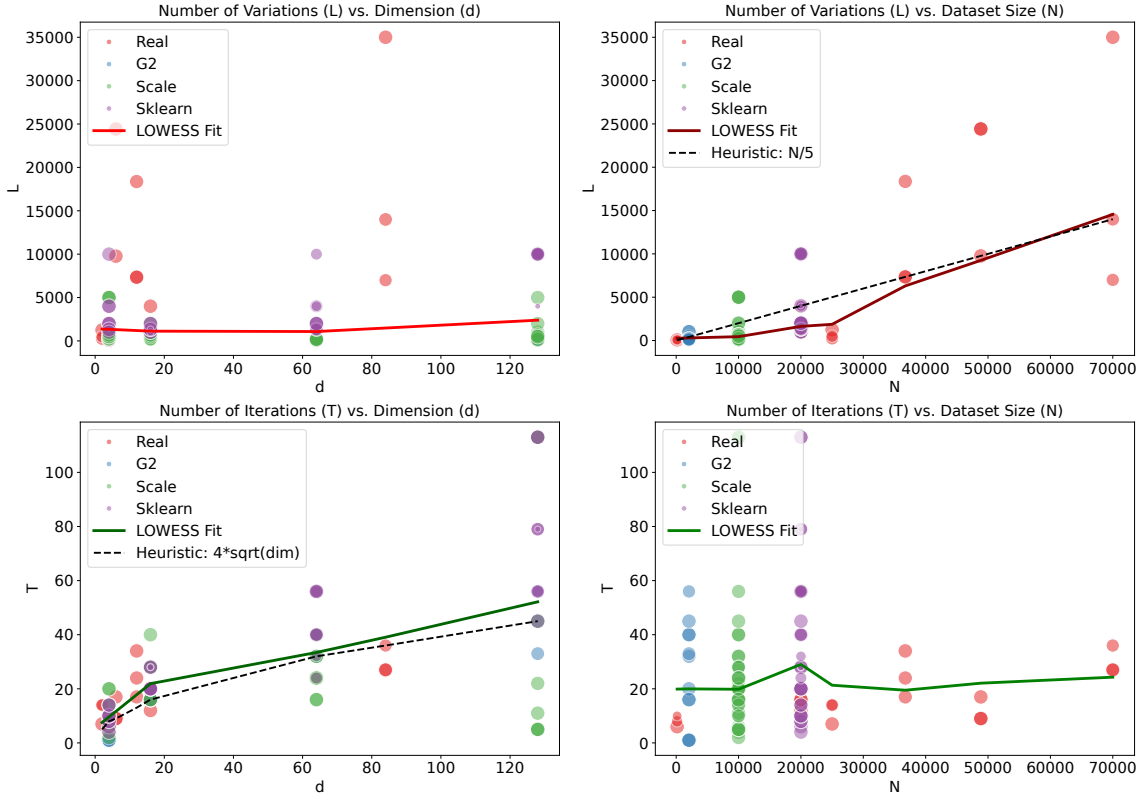


Figure 2: A scatter plot of the top-3 parameter configurations of L and T for each dataset at $\epsilon = 1$. The LOWESS line of best fit shows a clear relationship between L and N and between T and d that are captured by our heuristics.

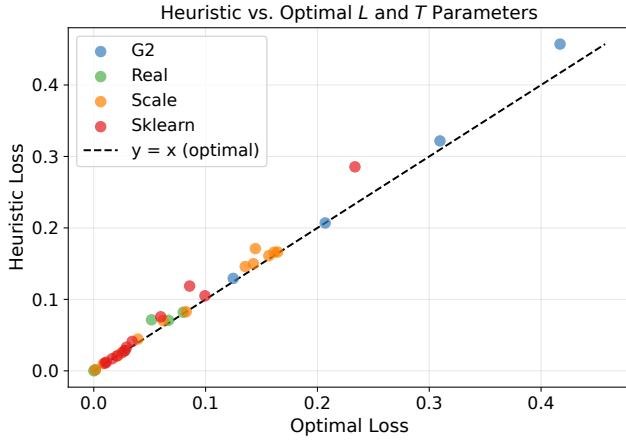


Figure 3: A scatter plot of grid search results over L and T with $\epsilon = 1$. There is one point per dataset with the x-axis shows the clustering loss of the optimal parameter setting in the grid and the y-axis showing the loss of our heuristic for that dataset. We observe that the loss of our heuristic parameter settings are close to the optimal loss for all datasets.

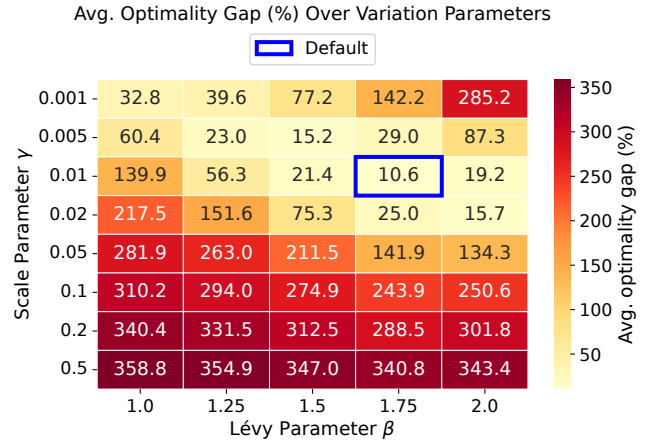


Figure 4: A heat map of the grid search results over γ and β at $\epsilon = 1$ with each cell showing the percentage increase in clustering loss for a given hyperparameters setting compared to the optimal hyperparameter choice for each dataset, averaged over all datasets. We observe our default parameter choice is the closest to the optimal loss.

Table 1: A summary of results over all datasets (real datasets highlighted in bold) showing the AUC of the clustering loss across ϵ values with the best approach highlighted for each dataset. The final column shows the relative improvements of PE-based techniques over the state-of-the art baseline for that dataset, with an average improvement of 26% over all datasets for PE-based approaches.

Dataset	N	d	k	Non-Priv	PE-means	HDPE-means	Google-LSH	FastLloyd	DP-Lib	Icml17	PE Improv.
birch2	25000	2	100	0.0001	0.0003	-	0.0658	0.0031	0.0124	0.0161	+90.92%
iris	150	4	3	0.1361	0.2894	-	0.6892	0.3979	1.3048	5.5766	+27.28%
adult	48842	6	3	0.0056	0.0056	-	0.0076	0.0155	0.0213	0.0113	+25.55%
mnist	70000	84	10	0.1958	0.2920	0.2169	0.2173	0.5822	0.5315	0.7904	+0.16%
letter	20000	16	26	0.2481	0.2852	-	0.3187	0.3963	0.4341	0.6277	+10.50%
gas	36733	12	6	0.1026	0.1081	-	0.1121	0.1561	0.1761	0.1853	+3.60%
g2_4	2048	4	2	0.4650	0.4761	-	0.5465	0.4666	0.5468	0.8802	-2.05%
g2_16	2048	16	2	0.7677	0.7800	-	0.8391	0.8687	1.1816	4.7489	+7.04%
g2_64	2048	64	2	1.0956	1.1538	1.1295	1.2478	1.9647	2.6608	27.5361	+9.49%
g2_128	2048	128	2	1.3534	1.7237	1.4055	1.5906	2.6425	4.4632	74.0978	+11.64%
scale_4_4	10000	4	4	0.0821	0.0814	-	0.1190	0.1272	0.1381	0.1772	+31.54%
scale_4_16	10000	16	4	0.3013	0.3147	-	0.3392	0.3721	0.4114	0.5948	+7.24%
scale_4_64	10000	64	4	0.4966	0.5326	0.5340	0.5430	0.5863	0.6622	2.5354	+1.92%
scale_4_128	9999	128	4	0.5731	0.6179	0.5940	0.5987	0.6282	0.7481	8.5885	+0.79%
scale_16_4	10000	4	16	0.0302	0.0335	-	0.0555	0.0864	0.0976	0.1525	+39.63%
scale_16_16	10000	16	16	0.2110	0.2452	-	0.2880	0.3624	0.3778	0.5679	+14.87%
scale_16_64	10000	64	16	0.4824	0.5884	0.5757	0.5823	0.6371	0.6949	2.5353	+1.14%
scale_16_128	10001	128	16	0.5410	0.6131	0.6023	0.5992	0.6294	0.7612	8.6815	-0.50%
scale_64_4	10002	4	64	0.0029	0.0051	-	0.0214	0.0306	0.0381	0.0847	+76.08%
scale_64_16	10001	16	64	0.1126	0.1607	-	0.1902	0.2393	0.2771	0.5238	+15.54%
scale_64_64	10001	64	64	0.4487	0.5518	0.5567	0.5453	0.5827	0.6913	2.5620	-1.20%
scale_64_128	10016	128	64	0.5206	0.6874	0.6232	0.6299	0.6421	1.0248	8.6433	+1.07%
sklearn_4_4	20000	4	4	0.0777	0.0778	-	0.1305	0.1777	0.2355	0.1240	+37.22%
sklearn_16_4	20000	4	16	0.0459	0.0445	-	0.0979	0.0990	0.1409	0.2601	+54.59%
sklearn_64_4	20000	4	64	0.0343	0.0379	-	0.1289	0.0818	0.1191	0.3227	+53.68%
sklearn_4_16	20000	16	4	0.0950	0.0955	-	0.1395	0.7110	0.5049	0.6074	+31.50%
sklearn_16_16	20000	16	16	0.0606	0.0668	-	0.1596	0.7160	0.5809	1.4670	+58.16%
sklearn_64_16	20000	16	64	0.0642	0.1183	-	0.3872	0.7631	0.8720	1.7704	+69.43%
sklearn_4_64	20000	64	4	0.0954	0.0993	0.0974	0.1090	1.4623	0.8614	2.7731	+10.60%
sklearn_16_64	20000	64	16	0.0847	0.1192	0.1189	0.2258	1.7475	1.3716	3.1765	+47.34%
sklearn_64_64	20000	64	64	0.0823	0.3070	0.4678	0.7878	2.2002	2.2456	3.1698	+61.03%
sklearn_4_128	20000	128	4	0.1054	0.1333	0.1097	0.1186	1.7604	1.1632	5.6663	+7.49%
sklearn_16_128	20000	128	16	0.0919	0.1942	0.1562	0.2838	2.3038	1.9742	4.9881	+44.97%
sklearn_64_128	20000	128	64	0.0933	0.5740	0.7190	0.9675	2.7204	2.9608	4.9597	+40.67%

4.3 Hyperparameter Tuning

4.3.1 Iterations and Variation Size. Two of the most influential hyperparameters for PE-means are the number of iterations T and the number of variations L . For a fixed privacy budget, if the number of iterations is too high, the amount of noise used in each iteration grows too large, preventing meaningful selections. If the algorithm executes too few iterations, it will not converge. Similarly, if L is too large, the histogram can become sparse and the votes will not dominate even the smallest amount of noise. If L is too small, then the chance of PE-means finding points closer to the private data is reduced, slowing convergence. We recall that our adaptive reduction of L (Line 8 of Algorithm 1) protects against the case when L is causing the noise to dominate the signal, but it is still beneficial to set a good starting point.

The only public parameters we can use to estimate the difficulty of a dataset in the private setting are its dimensions. In our initial testing, we observed that the number of iterations is influenced by

the dimension of the dataset. Namely, in higher-dimensional space, PE needs more steps due to the curse of dimensionality. We also observed that the sparsity of the histogram for a given L is highly influenced by the number of values in a dataset that can vote. To investigate this more rigorously, we conduct an experiment where we vary L and T for each dataset and plot the values against the dataset size and dimension. This experiment uses all datasets to give a variety of datasets spanning dimensions, dataset sizes, and values of k . We plot the results in Figure 2, where $\epsilon = 1$. We scatter the values of the top-3 best-performing parameters for each dataset and draw a line of best fit using the LOWESS method. While the top-left plot reveals no useful relationship between the number of variations L and the number of dimensions d , the top-right plot shows a clear relationship between the dataset size N and L . We approximate this relationship with the heuristic $L = \max(N/5, 4)$. Similarly, we see a clear relationship between the dimension d and number of iterations T in the bottom-left plot, which we approximate with the heuristic

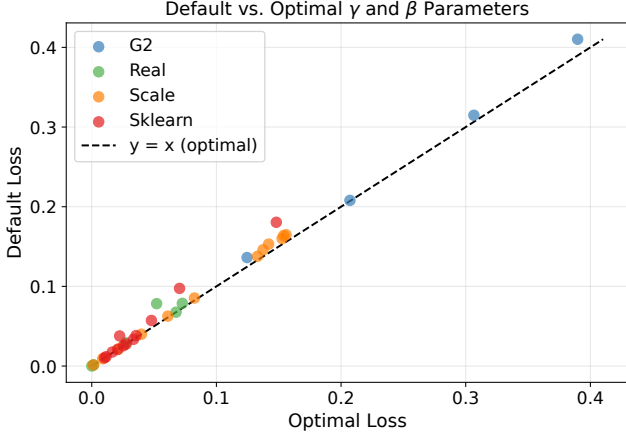


Figure 5: A scatter plot of grid search results over γ and β with $\epsilon = 1$. There is one point per dataset with the x-axis shows the clustering loss of the optimal parameter setting in the grid and the y-axis showing the loss of our default parameter values. We observe that the loss of our default parameter settings are close to the optimal loss for all datasets.

$T = \max(4\sqrt{d}, 1)$. Finally, we see no clear relationship between N and T in the bottom-right plot. In our experiments, we never encounter the constant cases in the max terms, but include them to ensure a robust implementation. While Figure 2 demonstrates that our heuristics closely follow the line of best fit, it is also clear that the optimal settings for a number of datasets are quite far away from the line. To further investigate this, we consider the difference in utility between the optimal settings and the heuristic settings in Figure 3. The x-axis represents the loss of the optimal configuration of T and L over the entire grid search, and the y-axis represents the loss when T and L are set following the heuristic. We can see most points lie on or very close to the line $y = x$, indicating near-optimality. Thus, we argue our heuristics offer good default values in practice.

To extend these heuristics to also account for the privacy budget ϵ , we experimented with scaling them by factors such as ϵ , 0.5ϵ , 2ϵ , $\sqrt{\epsilon}$ (since both parameters should decrease with smaller epsilon). We find that for values of $\epsilon < 1$, no modification improves over the baseline, as our adaptive strategy increases the signal-to-noise ratio for any dataset that needs it (and not all do). However, when $\epsilon > 1$, we find it is best (taking the median performance over all datasets) to scale the number of iterations by ϵ ($T = \max(4\epsilon\sqrt{d}, 1)$).

4.3.2 Variation Parameters. The remaining hyperparameters are those of our variation API. The first is the Lévy index β that determines how heavy the tails of the distribution are. Smaller values indicate a heavy tail with $\beta = 1$ representing a Cauchy distribution, and $\beta = 2$ representing a Gaussian. In our implementation, we add a condition to explicitly draw from a standard Gaussian at $\beta = 2$. The second parameter, the variation scale γ , is a multiplicative factor to control how much to vary the centroids. We conduct a grid search over possible values of each parameter (with L and T fixed to the heuristics from the previous section) and plot the results in

Figure 4. We plot the grid search results as a heatmap with each cell showing the percentage increase of this parameter configuration over the optimal parameter configuration, averaged over all datasets we evaluate (see Table 1 for the complete list of datasets). We can see that $\beta = 1.75$ and $\gamma = 0.01$ give the best results and thus, we choose these values as our default. In Figure 5, we scatter the optimal vs. default values of β and γ over all datasets. We can see that the default parameters stay close to the optimal line of $y = x$, indicating a good choice of parameters.

4.4 Utility Benchmark

Overall Comparison. We compare PE-means and HDPE-means to the baselines over all datasets and summarize the results in Table 1. The metric we use is the AUC of the k -means loss (a summary of performance over a collection of epsilons, with smaller values being best).⁵ We highlight the approach with the lowest loss AUC in green. HDPE-means is only included when the dimension of the dataset is greater than 16, as that is the dimensionality we project down to. Overall, PE-means and HDPE-means perform the best over most datasets, with FastLloyd doing well on the g2_4 dataset, and Google-LSH doing well in two of the high-dimensional scale datasets. We give the percentage of improvement (or decline) of the best PE-based approach compared to the best non-PE private approach in the last column. We see that when PE-based approaches are outperformed, it is by at most 2%, but on average, they improve by 26%, with improvements of as much as 91%. Finally, we note that HDPE-means typically outperforms PE-means on datasets with dimension larger than 16. In high-dimensional cases where PE-means outperforms HDPE-means, it is typically not by a significant amount compared to the performance of related work, and thus we recommend always using HDPE-means in $d > 16$ datasets.

Real Dataset Privacy vs. Utility Trade-off. We give the full privacy vs. utility trade-off for the various real datasets (including the three evaluated by Google-LSH [4]) in Figure 6. We observe that a PE-based approach is always the best approach, being the closest to the non-private baseline in all plots. Among the baselines, we observe that FastLloyd performs well in the lower dimensions (birch2 and iris) and Google-LSH does well in higher dimensions. We also note that our PE-based approaches scale well to limited privacy budgets.

Ground Truth Evaluation. Following Google-LSH [4], we also evaluate an alternate metric called cluster label accuracy. We only evaluate this metric for datasets for which we have a ground-truth labelling, namely: mnist, letter, iris, birch2, and the G2 datasets. Given a clustering of the private data, we compute the metric by first assigning each centroid a label through a majority vote of the ground-truth label of the points in that cluster. Then we simply compute accuracy based on the number of private points correctly labelled by the centroid label. The results are given in Figure 7. We observe that for the letter, iris, and birch2 datasets, PE-means outperforms related work by a statistically significant amount as the confidence intervals do not overlap. In the remaining datasets, PE-means or HDPE-means perform similarly to the best baselines as

⁵To convert the non-private k -means loss to an AUC score, we compute the area under the horizontal line drawn at the non-private utility over the same set of epsilons evaluated for the private approaches.

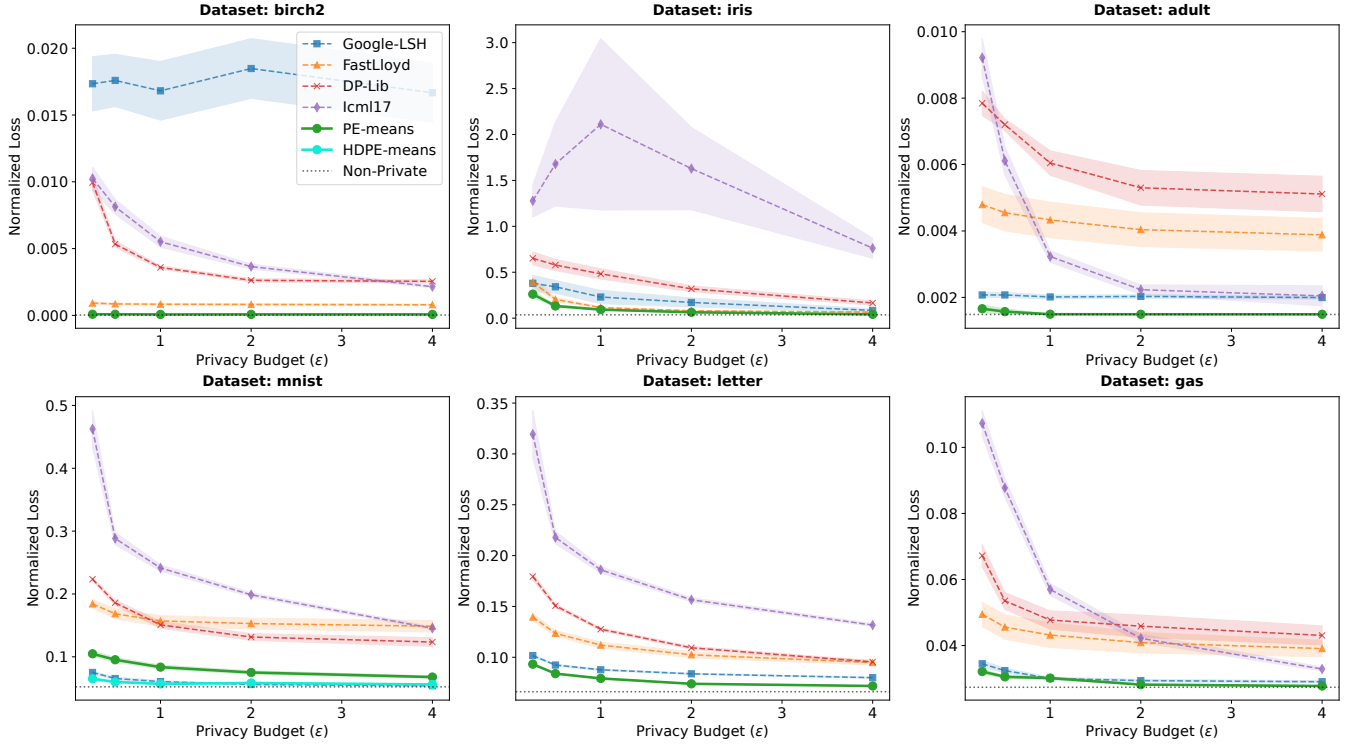


Figure 6: A plot of the privacy-utility trade-off of all approaches on real datasets, showing a PE-based approach always performs best.

well as the non-private baseline, indicating near-optimal performance.

Scaling Evaluation. In Figure 8, we show how the algorithms scale with the number of clusters and a fixed privacy budget of $\epsilon = 1$. We omit lcml17 from the scale dataset plots for clarity as the loss is an order of magnitude worse than other approaches. The scale datasets follow the trend of the non-private version until we increase dimensions, at which point the noise has more of an effect at higher numbers of clusters. This is likely because the data size remains fixed, causing the number of samples voting (or being aggregated) per cluster to decline. The scale datasets also highlight the room for improvement in PE-based approaches in higher dimensions. For the Sklearn sets, which tend to be more well-separated Gaussian blobs, we observe PE-based approaches outperforming all other approaches and also scaling better with k .

4.5 Discussion

In addition to showing the improvements of PE-based approaches in clustering, our experimental evaluations also provide support for the PE algorithm in general. Specifically, PE-means does not use foundation models to implement the random and variation APIs. This means we can rule out the effect of dataset contamination (evaluation datasets appearing in the foundation model’s training set) in our results. This gives additional evidence to support the effectiveness of the PE paradigm independently of the improvement of foundation models.

5 RELATED WORK

DP k -means. In addition to the baselines we compare against in Section 4, there are several previous works that also solve the DP k -means clustering problem, which are either improved upon by our baselines or are more theoretical in nature. One line of work focused on developing a private version of Lloyd’s algorithm [3, 5, 9, 25, 31] starting with Blum et al. [3] and concluding with our baselines of Su et al. [31] and FastLloyd [5]. Another line of work used the sample and aggregate framework [26, 27] to solve k -means, but was outperformed by Su et al. [31]. A theoretical line of work focused on minimizing the bounds on approximation error, but did not provide experimental evaluation [12, 13, 15, 19, 28, 30].

DP Evolutionary Algorithms. There has been previous work that has applied evolutionary techniques to the problem of clustering. The first differentially private evolutionary algorithm was developed by Zhang et al. [34], who evaluated it on the k -means clustering problem. Follow-up work by Humphries and Kerschbaum [18] showed that Zhang et al.’s solution suffered from prohibitively poor utility and provided an improved algorithm which was evaluated on the k -medians clustering problem. While our work also applies an evolutionary-based approach to solve clustering, our algorithm uses significantly less privacy budget. The population of PE-means is a set of centroids from which a single solution of k points is chosen in each iteration using a DP histogram with sensitivity 1, following the previous work in PE [21, 22, 32, 33]. Humphries and Kerschbaum instead evolve a population where each candidate is a solution of k

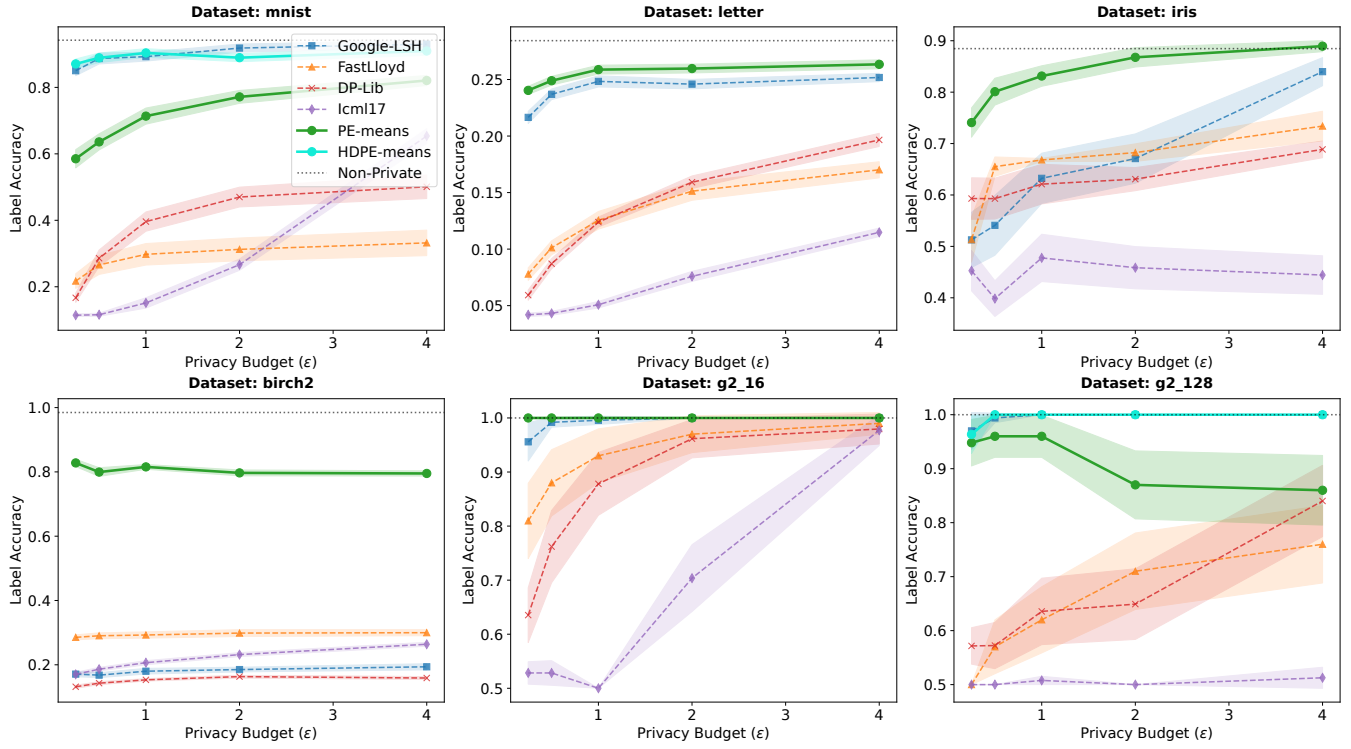


Figure 7: A plot of the label accuracy metric vs. privacy budget ϵ over all approaches on a subset of real datasets that have ground-truth labels. We observe that a PE-based approach always performs better or similar to state-of-the-art.

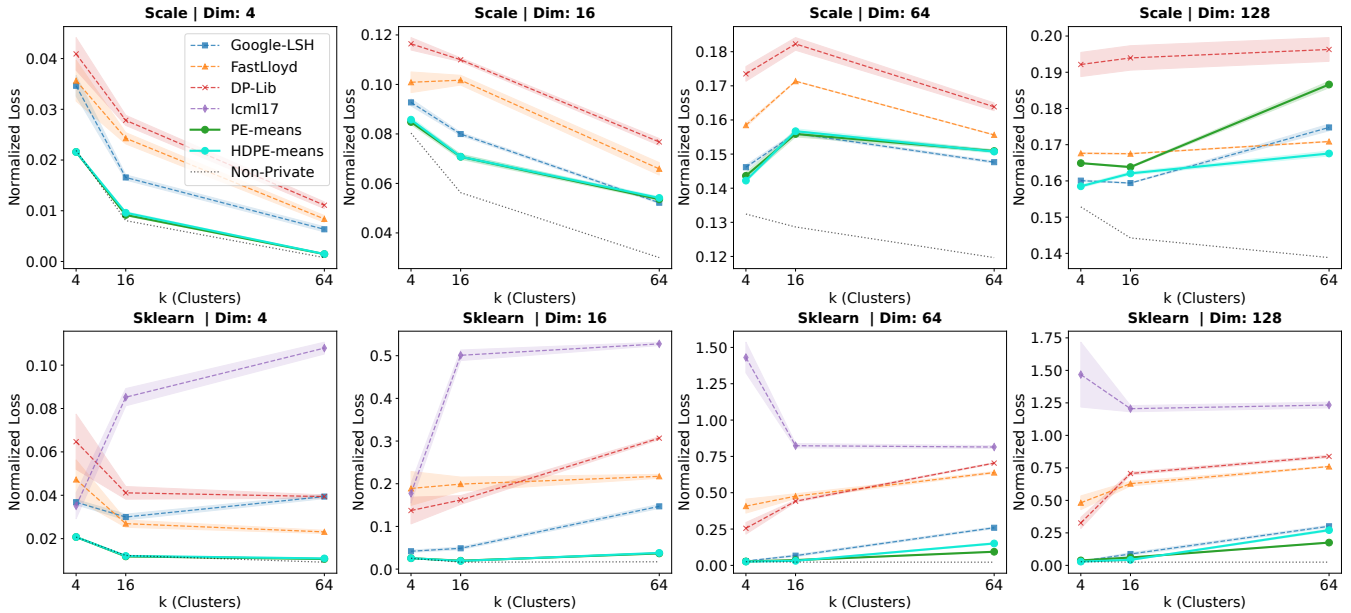


Figure 8: A comparison illustrating how approaches scale with the number of clusters k over different synthetic datasets at $\epsilon=1.0$.

points and apply the exponential mechanism on the clustering loss (with sensitivity proportional to the data domain) to select multiple candidates in each iteration, using significantly more privacy budget.

6 CONCLUSION

We have shown that the PE algorithm is well-suited to the problem of k -means clustering. Our benchmark shows that PE-means and HDPE-means offer state-of-the-art performance on many synthetic and real clustering datasets. Furthermore, the design of PE-means is of independent interest, offering solutions to vote-splitting and signal-to-noise ratio management that can be extended to other modalities of PE. In future work, we will study variation APIs that more efficiently search high-dimensional Euclidean spaces.

ACKNOWLEDGMENTS

We would like to thank Sivakanth Gopi for helpful discussions on an earlier version of this work, including the suggestion to use MLE post-processing on the vote histogram.

REFERENCES

- [1] Maria-Florina Balcan, Travis Dick, Yingyu Liang, Wenlong Mou, and Hongyang Zhang. 2017. Differentially Private Clustering in High-Dimensional Euclidean Spaces. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 70. PMLR, 322–331.
- [2] Borja Balle and Yu-Xiang Wang. 2018. Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Jennifer Dy and Andreas Krause (Eds.), Vol. 80. PMLR, 394–403. <https://proceedings.mlr.press/v80/balle18a.html>
- [3] Avrim Blum, Cynthia Dwork, Frank McSherry, and Kobbi Nissim. 2005. Practical privacy: the SuLQ framework. In *Proceedings of the twenty-fourth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems* (New York, NY, USA, 2005-06-13) (PODS '05). 128–138. <https://doi.org/10.1145/1065167.1065184>
- [4] Alisa Chang and Pritish Kamath. 2024. Practical differentially private clustering. <https://research.google/blog/practical-differentially-private-clustering>
- [5] Abdulrahman Diaa, Thomas Humphries, and Florian Kerschbaum. 2025. {FastLloyd}: Federated, accurate, secure, and tunable {k-Means} clustering with differential privacy. In *34th USENIX Security Symposium (USENIX Security 25)*. 2733–2752.
- [6] Jinshuo Dong, Aaron Roth, and Weijie J. Su. 2022. Gaussian Differential Privacy. *Journal of the Royal Statistical Society Series B: Statistical Methodology* 84, 1 (02 2022), 3–37. <https://doi.org/10.1111/rssb.12454>
- [7] Dheeru Dua and Casey Graff. 2021. UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml>
- [8] John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. 2008. Efficient projections onto the l_1 -ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning (Helsinki, Finland) (ICML '08)*. Association for Computing Machinery, New York, NY, USA, 272–279. <https://doi.org/10.1145/1390156.1390191>
- [9] Cynthia Dwork. 2011. A firm foundation for private data analysis. *Commun. ACM* 54, 1 (2011), 86–95.
- [10] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Theory of Cryptography*. 265–284.
- [11] Cynthia Dwork and Aaron Roth. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science* 9, 3–4 (2014), 211–407. <https://doi.org/10.1561/04000000042>
- [12] Dan Feldman, Amos Fiat, Haim Kaplan, and Kobbi Nissim. 2009. Private coresets. In *Proceedings of the forty-first annual ACM symposium on Theory of computing* (New York, NY, USA, 2009-05-31) (STOC '09). Association for Computing Machinery, 361–370. <https://doi.org/10.1145/1536414.1536465>
- [13] Dan Feldman, Chongyuan Xiang, Ruihao Zhu, and Daniela Rus. 2017. Coresets for differentially private k -means clustering and applications to privacy in mobile sensor networks. In *Proceedings of the 16th ACM/IEEE International Conference on Information Processing in Sensor Networks* (Pittsburgh Pennsylvania, 2017-04-18). ACM, 3–15. <https://doi.org/10.1145/3055031.3055090>
- [14] Pasi Fränti and Sami Sieranoja. 2018. K -means properties on six clustering benchmark datasets. , 4743–4759 pages. <http://cs.uef.fi/sipu/datasets/>
- [15] Badih Ghazi, Ravi Kumar, and Pasin Manurangsi. 2020. Differentially Private Clustering: Tight Approximation Ratios. In *Advances in Neural Information Processing Systems* (2020), Vol. 33. Curran Associates, Inc., 4040–4054. https://proceedings.neurips.cc/paper_files/paper/2020/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html
- [16] Attri Ghosal, Arunima Nandy, Amit Kumar Das, Saptarsi Goswami, and Mritunjay Panday. 2020. A short review on different clustering techniques and their applications. *Emerging Technology in Modelling and Graphics: Proceedings of IEM Graph 2018* (2020), 69–83.
- [17] Naoise Holohan, Stefano Braghin, Pól Mac Aonghusa, and Killian Levacher. 2019. Diffprivlib: the IBM differential privacy library. *ArXiv e-prints* 1907.02444 [cs.CR] (July 2019).
- [18] Thomas Humphries and Florian Kerschbaum. 2023. Differentially Private Simple Genetic Algorithms. In *Proceedings on Privacy Enhancing Technologies (PoPETs)*. 540–558. Issue 4. <https://doi.org/10.56553/popets-2023-0124>
- [19] Matthew Jones, Huy L. Nguyen, and Thy D Nguyen. 2021. Differentially Private Clustering via Maximum Coverage. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 13 (May 2021), 11555–11563. <https://ojs.aaai.org/index.php/AAAI/article/view/17375>
- [20] Yann LeCun. 1998. The MNIST database of handwritten digits. <http://yann.lecun.com/exdb/mnist/> (1998).
- [21] Zinan Lin, Tadas Baltrušaitis, and Sergey Yekhanin. 2025. Differentially Private Synthetic Data via APIs 3: Using Simulators Instead of Foundation Model. In *ICLR 2025 Workshop on Navigating and Addressing Data Problems for Foundation Models*.
- [22] Zinan Lin, Sivakanth Gopi, Janardhan Kulkarni, Harsha Nori, and Sergey Yekhanin. 2024. Differentially Private Synthetic Data via Foundation Model APIs 1: Images. In *The Twelfth International Conference on Learning Representations*.
- [23] S. Lloyd. 1982. Least squares quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (March 1982), 129–137. <https://doi.org/10.1109/tit.1982.1056489>
- [24] Rosario Nunzio Mantegna. 1994. Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes. *Phys. Rev. E* 49 (May 1994), 4677–4683. Issue 5. <https://doi.org/10.1103/PhysRevE.49.4677>
- [25] Frank D. McSherry. 2009. Privacy Integrated Queries: An Extensible Platform for Privacy-Preserving Data Analysis. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD '09)*. Association for Computing Machinery, New York, NY, USA, 19–30. <https://doi.org/10.1145/1559845.1559850>
- [26] Prashanth Mohan, Abhradeep Thakurta, Elaine Shi, Dawn Song, and David Culler. 2012. GUPT: privacy preserving data analysis made easy. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. ACM, 349–360.
- [27] Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2007. Smooth sensitivity and sampling in private data analysis. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing (STOC '07)*. Association for Computing Machinery, New York, NY, USA, 75–84. <https://doi.org/10.1145/1250790.1250803>
- [28] Kobbi Nissim and Uri Stemmer. 2018. Clustering Algorithms for the Centralized and Local Models. In *Proceedings of Algorithmic Learning Theory* (2018-04-09). PMLR, 619–653. <https://proceedings.mlr.press/v83/nissim18a.html>
- [29] Weiliang Qiu and Harry Joe. 2023. *Random Cluster Generation (with Specified Degree of Separation)*. <https://CRAN.R-project.org/package=clusterGeneration> R package version 1.3.8.
- [30] Uri Stemmer and Haim Kaplan. 2018. Differentially Private k -Means with Constant Multiplicative Error. In *Advances in Neural Information Processing Systems* (2018), Vol. 31. Curran Associates, Inc. https://papers.nips.cc/paper_files/paper/2018/hash/32b991e5d77ad140559ffb95522992d0-Abstract.html
- [31] Dong Su, Jianneng Cao, Ninghui Li, Elisa Bertino, and Hongxia Jin. 2016. Differentially Private K -Means Clustering. In *Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy* (New Orleans, Louisiana, USA) (CODASPY '16). 26–37. <https://doi.org/10.1145/2857705.2857708>
- [32] Toan Tran, Arturs Backurs, Zinan Lin, Victor Reis, Li Xiong, and Sergey Yekhanin. 2026. Differentially Private Synthetic Data via APIs 4: Tabular Data. *arXiv preprint arXiv:2606.08259* (2026).
- [33] Chulin Xie, Zinan Lin, Arturs Backurs, Sivakanth Gopi, Da Yu, Huseyin A Inan, Harsha Nori, Haotian Jiang, Huishuai Zhang, Yin Tat Lee, Bo Li, and Sergey Yekhanin. 2024. Differentially Private Synthetic Data via Foundation Model APIs 2: Text. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 235. PMLR, 54531–54560.
- [34] Jun Zhang, Xiaokui Xiao, Yin Yang, Zhenjie Zhang, and Marianne Winslett. 2013. PrivGene: Differentially Private Model Fitting Using Genetic Algorithms. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2463676.2465330>
- [35] T. Zhang, R. Ramakrishnan, and M. Livny. 1997. BIRCH: A new data clustering algorithm and its applications. *Data Mining and Knowledge Discovery* 1, 2 (1997), 141–182.